

RunnableWithMessageHistory, Sessions & LCEL

1. What Is a Session?

A **session** represents one conversation context that should have its own history.

Example:

Session: durga



Durga's History

Session: ravi



Ravi's History

So:

One Session



One Separate Conversation Context

2. What Is session_id?

session_id is an identifier used to select the correct conversation history.

Example:

```
session_id = "durga"
```

means:

Use Durga Conversation History

Another example:

```
session_id = "ravi"
```

means:

Use Ravi Conversation History

By default, `RunnableWithMessageHistory` expects a string `session_id` inside its configuration and uses that value to create or retrieve the appropriate chat history.

3. Easy Memory Rule

SAME session_id



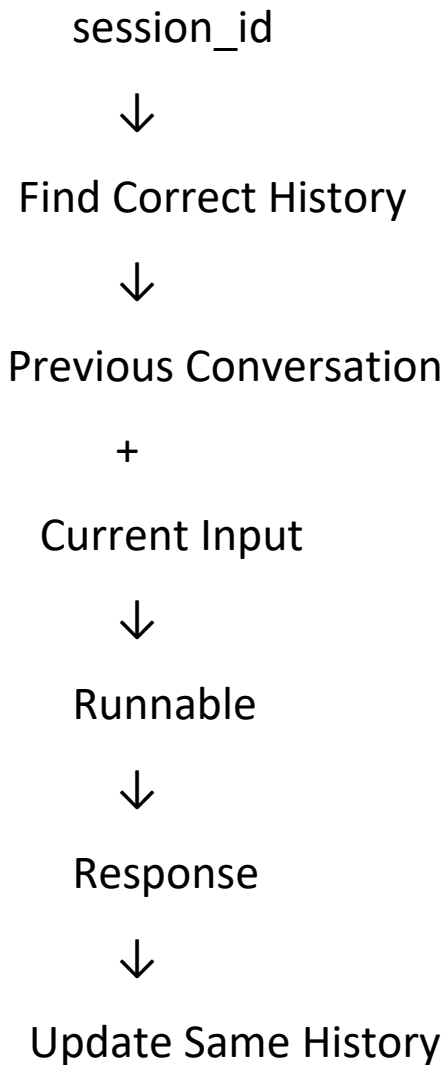
SAME HISTORY

DIFFERENT session_id



DIFFERENT HISTORY

4. Main Architecture



5. Main Components

For a simple session-based chatbot, we need:

1. Chat Model

2. `InMemoryChatMessageHistory`

3. History Store

4. `get_session_history()`

5. `RunnableWithMessageHistory`

6. `session_id`

6. Basic Session-Based Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
from langchain_core.chat_history import (  
    InMemoryChatMessageHistory  
)
```

```
from langchain_core.runnables.history import (  
    RunnableWithMessageHistory
```

)

=====

CREATE MODEL

=====

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

=====

STORE

=====

```
store = {}
```

```
# =====
```

```
# GET SESSION HISTORY
```

```
# =====
```

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
        store[session_id] = (  
            InMemoryChatMessageHistory()  
        )
```

```
    return store[session_id]
```

```
# =====
```

```
# WRAP MODEL WITH MESSAGE HISTORY
```

```
# =====
```

```
chatbot = RunnableWithMessageHistory(
```

```
    llm,  
    get_session_history  
)
```

7. What Is store?

We create:

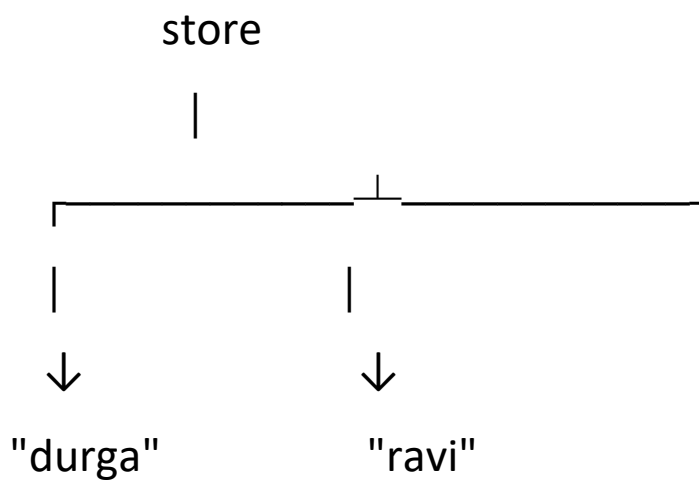
```
store = {}
```

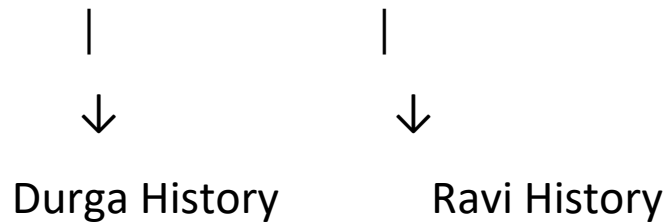
This dictionary can hold separate histories.

Conceptually:

```
store = {  
    "durga": DurgaHistory,  
    "ravi": RaviHistory  
}
```

Architecture:





8. What Does `get_session_history()` Do?

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
        store[session_id] = (  
            InMemoryChatMessageHistory()  
        )
```

```
    return store[session_id]
```

This function performs two jobs:

Check whether history exists



If not, create history



Return correct history

9. Flow of get_session_history()

session_id



Check store



History exists?

/ \

No

Yes



Create

Reuse

New History Existing History

\ /



Return History

10. First Session Example

```
response = chatbot.invoke(
```

```
[
```

```
    HumanMessage(
```

```
        content="My name is Durga."  
    )  
],
```

```
config={  
    "configurable": {  
        "session_id": "durga"  
    }  
}  
)
```

This means:

Current Message

"My name is Durga."



session_id = "durga"



Durga History

11. Continue the Same Session

```
response = chatbot.invoke(  
  
    [  
        HumanMessage(  
            content="I am teaching Python."  
        )  
    ],  
  
    config={  
        "configurable": {  
            "session_id": "durga"  
        }  
    }  
)
```

Because we again use:

```
"session_id": "durga"
```

the same Durga history is used.

So:

First Durga Message



Stored in Durga Session

Second Durga Message



Same Durga Session

12. Start a Different Session

```
response = chatbot.invoke(
```

```
[
    HumanMessage(
        content="I am Ravi."
    )
],
```

```
config={
    "configurable": {
        "session_id": "ravi"
    }
}
```

```
}  
)
```

Now:

```
session_id = "ravi"
```



Ravi History

This is separate from:

Durga History

13. Multiple Conversations Example

```
response = chatbot.invoke(  
    [  
        HumanMessage(  
            content="My name is Durga."  
        )  
    ],  
    config={  
        "configurable": {  
            "session_id": "durga"  
        }  
    })
```

```
}  
)
```

```
response = chatbot.invoke(  
    [  
        HumanMessage(  
            content="I am teaching Python."  
        )  
    ],  
    config={  
        "configurable": {  
            "session_id": "durga"  
        }  
    }  
)
```

```
response = chatbot.invoke(  
    [  

```

```
HumanMessage(  
    content="I am Ravi."  
)  
,  
config={  
    "configurable": {  
        "session_id": "ravi"  
    }  
}  
)
```

```
response = chatbot.invoke(  
    [  
        HumanMessage(  
            content="I like movies."  
        )  
    ],  
    config={  
        "configurable": {
```

```
        "session_id": "ravi"  
    }  
}  
)
```

```
response = chatbot.invoke(  
    [  
        HumanMessage(  
            content="I am from Bangalore."  
        )  
    ],  
    config={  
        "configurable": {  
            "session_id": "ravi"  
        }  
    }  
)
```

14. Store After Multiple Conversations

Conceptually:

store

|

└─ "durga"

| ↓

| HumanMessage(...)

| AIMessage(...)

| HumanMessage(...)

| AIMessage(...)

|

└─ "ravi"

 ↓

 HumanMessage(...)

 AIMessage(...)

 HumanMessage(...)

 AIMessage(...)

Each session maintains its own message history.

15. Inspect Available Sessions

```
print("AVAILABLE SESSIONS")
```

```
print("-----")
```

```
print(store.keys())
```

Possible output:

```
dict_keys(['durga', 'ravi'])
```

This tells us which sessions currently exist.

16. Inspect One Session History

```
history = store[
```

```
    "durga"
```

```
]
```

```
for message in history.messages:
```

```
    print(
```

```
        type(message).__name__,
```

```
        ":",
```

```
        message.content
```

```
)
```

This prints only Durga's conversation history.

17. Inspect Session History — Complete Example

```
print("AVAILABLE SESSIONS")
```

```
print("-----")
```

```
print(store.keys())
```

```
print("\nSESSION 1 HISTORY")
```

```
print("-----")
```

```
history = store[
```

```
    "durga"
```

```
]
```

```
for message in history.messages:
```

```
print(  
    type(message).__name__,  
    ":",  
    message.content  
)
```

18. Current Approach with an LCEL Chain

We already know:

```
chain = prompt | llm
```

This is a Runnable.

Now the question is:

Can we add message history to this LCEL chain?

Yes.

We can wrap the chain using:

`RunnableWithMessageHistory`

19. LCEL + Conversation History

Concept:

prompt | llm



Runnable



RunnableWithMessageHistory



History-Aware Runnable

20. Prompt Architecture

```
prompt = ChatPromptTemplate.from_messages(
```

```
[
    (
        "system",
        "You are a helpful assistant."
    ),
```

```
    MessagesPlaceholder(
        variable_name="history"
```

```
    ),  
  
    (  
        "human",  
        "{question}"  
    )  
]  
)
```

The prompt contains:

System Message



Conversation History



Current Human Question

21. What Is MessagesPlaceholder?

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

This provides a position inside the prompt for previous conversation messages.

Concept:

Prompt

```
|  
|— System Message  
|  
|— Historical Messages  
|  
|— Current Human Message
```

22. Dictionary-Input Version

When the wrapped Runnable accepts dictionary input, we can write:

```
chain_with_history = RunnableWithMessageHistory(  
    chain,  
    get_session_history,  
    input_messages_key="question",  
    history_messages_key="history"  
)
```

Two names are very important:

input_messages_key

history_messages_key

23. What Is input_messages_key?

input_messages_key="question"

means:

Which input field contains the current user message?

Our invocation may look like:

```
{  
  "question": "What is my name?"  
}
```

So:

question



Current User Input

24. Easy Meaning

`input_messages_key`



Which key contains
the latest user input?

Example:

`input_messages_key="question"`

matches:

```
{  
    "question": "Explain Python"  
}
```

25. What Is `history_messages_key`?

`history_messages_key="history"`

means:

Which prompt variable should receive previous conversation messages?

Our prompt contains:

```
MessagesPlaceholder(  
    variable_name="history"
```

)

So these names match:

history_messages_key

↓

"history"

↓

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

Very very important.

26. Three Important Names

Students should carefully observe:

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

```
input_messages_key="question"
```

```
history_messages_key="history"
```

Remember:

```
question
```



Current Input

history



Previous Messages

27. LCEL + RunnableWithMessageHistory — Complete Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import (
    ChatPromptTemplate,
    MessagesPlaceholder
)
```

```
from langchain_core.chat_history import (
    InMemoryChatMessageHistory
)
```

```

from langchain_core.runnables.history import (
    RunnableWithMessageHistory
)

# =====
# CREATE MODEL
# =====

llm = ChatOpenAI(
    model="gpt-5.6-luna",
    api_key=os.getenv("OPENAI_API_KEY")
)

# =====
# CREATE CHAT PROMPT
# =====

prompt = ChatPromptTemplate.from_messages(

```

```
[
    (
        "system",
        "You are a helpful AI assistant."
    ),

    MessagesPlaceholder(
        variable_name="history"
    ),

    (
        "human",
        "{question}"
    )
]
)

# =====
# CREATE LCEL CHAIN
```

```
# =====
```

```
chain = (  
    prompt  
    | llm  
)
```

```
# =====
```

```
# SESSION HISTORY STORE
```

```
# =====
```

```
store = {}
```

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
        store[session_id] = (
```

```

        InMemoryChatMessageHistory()
    )

    return store[session_id]

# =====
# WRAP LCEL CHAIN WITH HISTORY
# =====

chatbot = RunnableWithMessageHistory(

    chain,

    get_session_history,

    input_messages_key="question",

    history_messages_key="history"
)

```

```
# =====
```

```
# FIRST MESSAGE
```

```
# =====
```

```
response1 = chatbot.invoke(
```

```
{
```

```
    "question":
```

```
    "My name is Durga."
```

```
},
```

```
config={
```

```
    "configurable": {
```

```
        "session_id": "session1"
```

```
    }
```

```
}
```

```
)
```

```
print(
```

```
"AI:",  
response1.content  
)
```

```
# =====  
# SECOND MESSAGE  
# =====
```

```
response2 = chatbot.invoke(  
  

```

```
{  
    "question":  
        "What is my name?"  
},
```

```
config={  
    "configurable": {  
        "session_id": "session1"  
    }  
}
```

```
}  
)
```

```
print(  
    "AI:",  
    response2.content  
)
```

28. Understanding the LCEL Flow

First call:

"My name is Durga."



question



Prompt



Model



Response



Stored in session1

Second call:

"What is my name?"



session1



Load Previous History



Prompt Receives:

System Message

+

History

+

Current Question

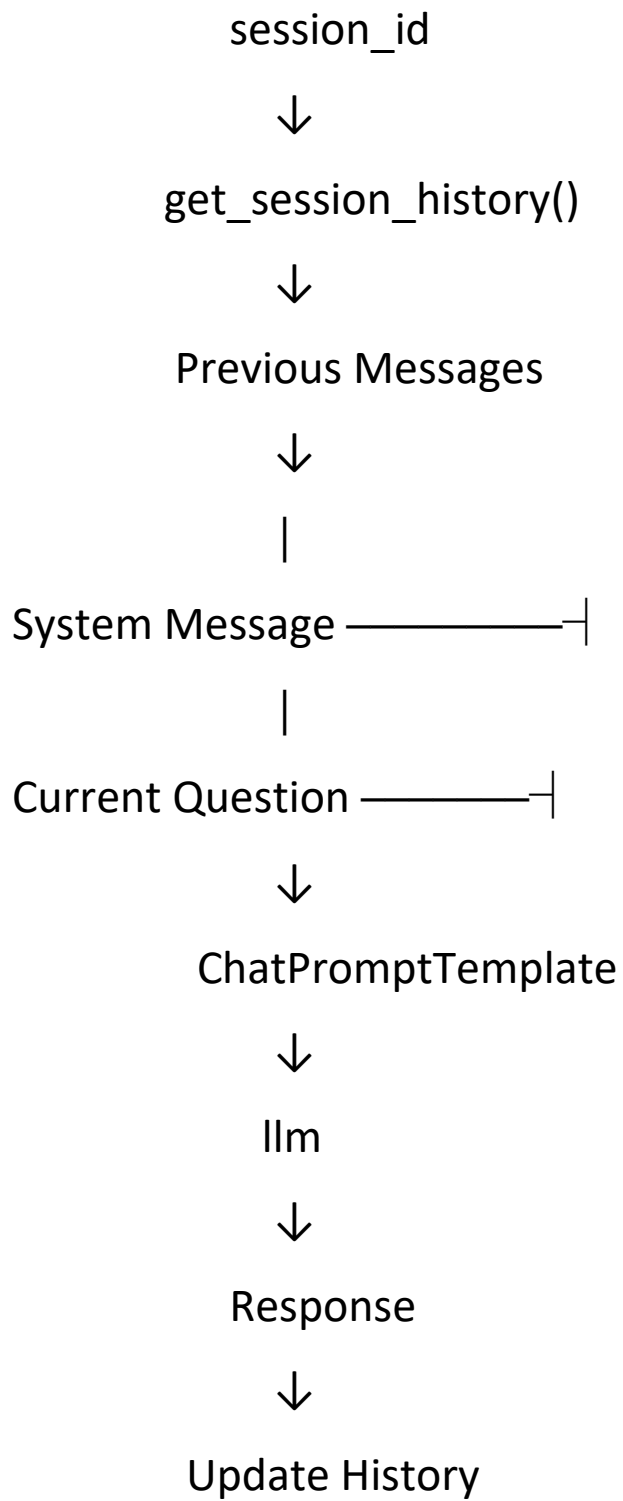


Model



Context-Aware Response

29. Main LCEL Architecture



30. Why Do the Names Need to Match?

Prompt contains:

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

Wrapper contains:

```
history_messages_key="history"
```

Therefore:

```
"history"
```

```
=
```

```
"history"
```

The wrapper knows where to inject previous messages.

Similarly:

```
input_messages_key="question"
```

matches the input:

```
{  
    "question": user_input  
}
```

31. Easy Connection Diagram

Dictionary Input

```
{  
  "question": "..."  
}
```

|

↓

input_messages_key
= "question"

Stored History

|

↓

history_messages_key
= "history"

|

↓

MessagesPlaceholder
= "history"

32. Interactive Session-Based Chatbot

Now we can allow the user to enter a session ID and continue a conversation.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import (
    ChatPromptTemplate,
    MessagesPlaceholder
)
```

```
from langchain_core.chat_history import (
    InMemoryChatMessageHistory
)
```

```
from langchain_core.runnables.history import (
    RunnableWithMessageHistory
)
```

```
# =====
```

```
# CREATE MODEL
```

```
# =====
```

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
# =====
```

```
# CREATE PROMPT
```

```
# =====
```

```
prompt = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            """"  
  
            You are a helpful AI assistant.
```

Answer in simple English.

""

),

MessagesPlaceholder(

variable_name="history"

),

(

"human",

"{question}"

)

]

)

=====

CREATE LCEL CHAIN

=====

```
chain = (  
    prompt  
    | llm  
)
```

```
# =====  
# SESSION HISTORY STORE  
# =====
```

```
store = {}
```

```
def get_session_history(session_id):
```

```
    if session_id not in store:
```

```
        store[session_id] = (  
            InMemoryChatMessageHistory()  
        )
```

```

return store[session_id]

# =====
# HISTORY-AWARE RUNNABLE
# =====

chatbot = RunnableWithMessageHistory(

    chain,

    get_session_history,

    input_messages_key="question",

    history_messages_key="history"
)

# =====

```

```
# SELECT SESSION
```

```
# =====
```

```
session_id = input(  
    "Enter Session ID: "  
)
```

```
print("\nAI Chatbot")  
print("Type 'exit' to stop")  
print("-----")
```

```
# =====
```

```
# CHAT LOOP
```

```
# =====
```

```
while True:
```

```
    user_input = input(
```

```
"\nYou: "  
)
```

```
if user_input.lower() == "exit":
```

```
    print("Chat ended.")
```

```
    break
```

```
response = chatbot.invoke(  
  
    {  
        "question": user_input  
    },  
  
    config={  
        "configurable": {  
            "session_id": session_id
```

```
    }  
  }  
)
```

```
print(  
    "AI:",  
    response.content  
)
```

33. Interactive Chatbot Flow

Enter Session ID



Choose Conversation



Read User Question



Use Same Session ID



Find Session History



Add History to Prompt



Call Model



Save Response



Next Question

34. Example — Session 1

User starts program and enters:

Enter Session ID: student1

Then:

You: My name is Durga.

Later:

You: What is my name?

Because both calls use:

student1

the same conversation history is available.

35. Example — Different Session

Suppose another conversation uses:

```
session_id = "student2"
```

Then:

student1



History A

student2



History B

The two conversations remain separate.

36. 15 Important Key Points

1. Conversation history is still a sequence of Human and AI messages.
2. InMemoryChatMessageHistory stores chat messages in application memory.
3. In-memory history is not automatically permanent storage.
4. Different conversations should normally have separate histories.

5. A session represents a particular conversation context.
6. `session_id` identifies which conversation history should be used.
7. Same `session_id` allows subsequent calls to use the same conversation history.
8. Different `session_id` values allow separate conversation histories.
9. `RunnableWithMessageHistory` wraps another `Runnable`.
10. The wrapper reads and updates the selected chat history.
11. A normal LCEL chain such as:

`prompt | llm`

can be wrapped with message history.

12. `MessagesPlaceholder` provides a position in the prompt for historical messages.
13. `input_messages_key` identifies the current-input field when using dictionary input.
14. `history_messages_key` identifies where historical messages belong in the wrapped `Runnable`'s input.
15. This architecture separates the `Runnable`, history storage, and session selection more explicitly than the older `ConversationChain` approach.

37. Five Coding Lines Students Must Remember

1. History

```
history = InMemoryChatMessageHistory()
```

2. History Store

```
store = {}
```

3. Session Function

```
def get_session_history(session_id):
```

4. History Wrapper

```
chatbot = RunnableWithMessageHistory(  
    chain,  
    get_session_history,  
    input_messages_key="question",  
    history_messages_key="history"  
)
```

5. Session Invocation

```
config={  
  "configurable": {  
    "session_id": "session1"  
  }  
}
```

38. Master Coding Pattern

STORE



store = {}

SESSION FUNCTION



get_session_history(session_id)

RUNNABLE



prompt | llm

WRAPPER



RunnableWithMessageHistory(...)

SESSION



configurable → session_id

39. Compare Legacy and Runnable-Based Approach

Legacy Approach	Runnable-Based Approach
ConversationChain	LCEL Runnable
ConversationBufferMemory	InMemoryChatMessageHistory
More automatic older abstraction	More explicit history architecture
Older API style	Runnable-based style
Conversation handled inside chain	History wrapper around Runnable
Less explicit session separation	Explicit session_id

40. Main Evolution

Manual History



`messages.append(...)`



Legacy Memory



ConversationBufferMemory

ConversationChain



Runnable-Based History



InMemoryChatMessageHistory

RunnableWithMessageHistory

`session_id`

41. Common Mistake — Same Model Means Same Conversation

Wrong:

Same Ilm



Same History

No.

The session decides the history.

Correct:

Same Ilm

+

session_id = "A"



History A

Same Ilm

+

session_id = "B"



History B

42. Common Mistake — session_id Means User Name

Not necessarily.

A session_id is a conversation identifier.

It may be:

"durga"

"session1"

"chat_1001"

"customer_25"

The important point is that it identifies the required conversation history.

43. Common Mistake — In-Memory Means Permanent

Wrong.

InMemoryChatMessageHistory

means:

Stored in Application Memory

It does not automatically mean:

Permanent Database Storage

44. Common Mistake — Wrong Input Key

If we define:

```
input_messages_key="question"
```

our invocation should provide:

```
{  
    "question": user_input  
}
```

Using an unrelated key can break the expected mapping.

45. Common Mistake — Wrong History Key

Prompt:

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

Wrapper:

```
history_messages_key="history"
```

These should refer to the same history field.

46. MCQ 1

What is the main purpose of:

RunnableWithMessageHistory

- A. Generate embeddings
- B. Manage message history around a Runnable
- C. Parse JSON
- D. Run parallel chains

Answer: B

Conclusion

It wraps another Runnable and manages its associated chat history.

47. MCQ 2

What happens when two calls use the same:
session_id

- A. They must use different models
- B. They can use the same conversation history
- C. History is deleted
- D. Pydantic validation starts

Answer: B

Conclusion

Same session ID selects the same session history.

48. MCQ 3

What happens when:

`session_id = "A"`

and:

`session_id = "B"`

are used?

- A. Both must share exactly the same history
- B. They can represent separate conversation histories
- C. Model is retrained twice
- D. RunnableSequence stops working

Answer: B

Conclusion

Different session IDs can keep conversations separated.

49. MCQ 4

What is:

```
MessagesPlaceholder(  
    variable_name="history"  
)
```

used for?

- A. API key
- B. Model name

- C. Position for historical messages in a chat prompt
- D. Token counting

Answer: C

Conclusion

MessagesPlaceholder provides a place in the prompt for previous conversation messages.

50. MCQ 5

Does:

InMemoryChatMessageHistory()

automatically mean permanent database storage?

- A. Yes
- B. No

Answer: B

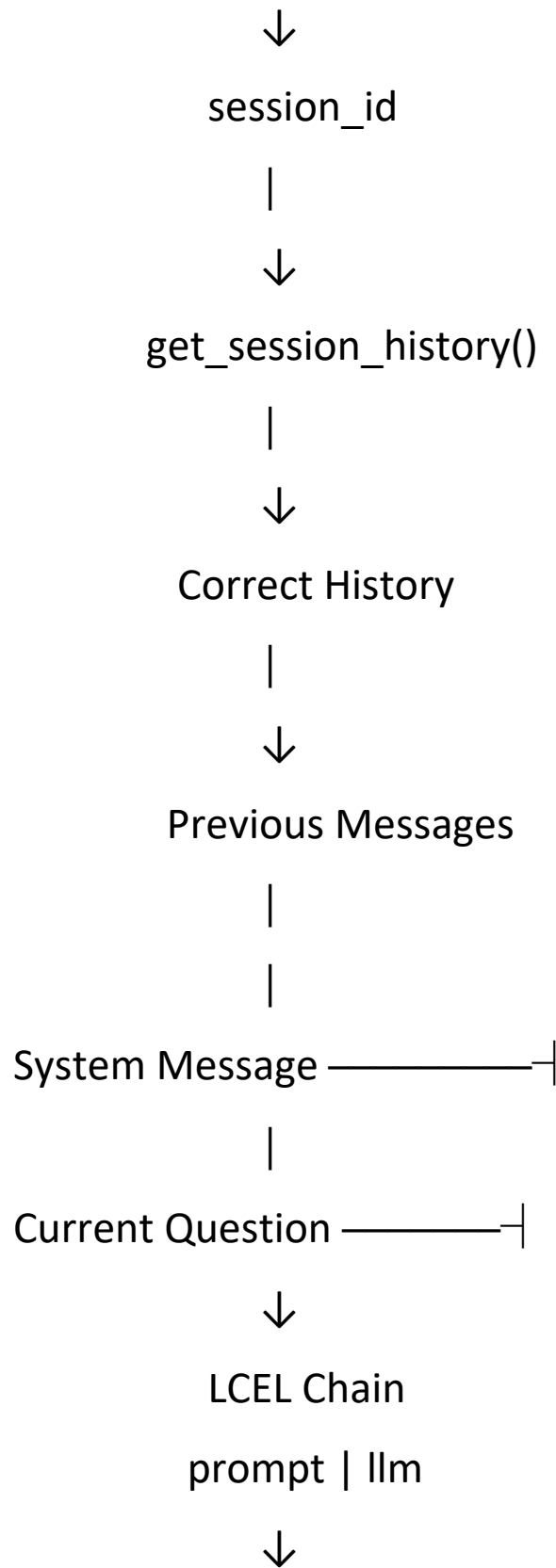
Conclusion

It stores history in application memory; persistence is a separate concern.

51. Final Master Architecture

USER INPUT

|



Response



RunnableWithMessageHistory



Update Same Session

52. Easy Memory Formula

SESSION ID



SELECT HISTORY



ADD HISTORY TO RUNNABLE



EXECUTE



UPDATE SAME HISTORY

53. Final Memory Rule

Same Session



Same History

Different Session



Different History

And:

Runnable

+

Message History

+

Session ID



History-Aware Conversation

54. One-Line Conclusion

RunnableWithMessageHistory wraps a **Runnable** with conversation-history management, while **session_id** selects the correct conversation history so that multiple conversations can remain separate.